

Watt Wallet Buddy: Comprehensive Technical Blueprint

Watt Wallet Buddy By UM TECH CG (Ubuntu Mzansi Tech Consulting Group)

This document provides a comprehensive technical blueprint for the **Watt Wallet Buddy** platform, a smart electricity wallet designed for the South African market with global scaling ambitions. It covers the architectural design, backend implementation details, frontend strategy, security and compliance considerations, scaling strategies, and a vision for future innovations. The goal is to establish a robust, scalable, secure, and maintainable foundation that is production-ready and attractive to investors.

Product Overview

Watt Wallet Buddy is a smart electricity wallet platform that empowers users to:

- Purchase prepaid electricity.
- Access electricity advances through a credit system.
- Manage shared household electricity usage efficiently.
- Control and approve electricity purchases within a household.
- Earn rewards ("white coins") based on responsible repayments and system usage.

Core Features

1. Electricity Wallet

Users can directly purchase electricity, with transactions generating prepaid tokens. Future enhancements include auto-loading via smart meters.

2. Advance (Credit) System

This system allows users to request electricity in advance, incorporating a service fee, VAT, and comprehensive tracking of total repayment.

3. Multi-User Meter System

Each electricity meter is managed by a **Primary account holder** (with full control and liability) and can have **Secondary users** with limited, configurable permissions.

4. Approval Workflow (Critical Feature)

Secondary users can request electricity advances, but the Primary user must explicitly approve or decline these requests. This critical feature prevents misuse and ensures accountability.

5. Rewards System ("White Coins")

Users earn rewards ("white coins") for repaying advances and responsible system usage.

6. Notifications

The platform provides notifications for advance requests, approvals/declines, and will include low electricity alerts in the future.

Current Tech Stack

- **Backend:** Node.js (Express)
- **Database:** PostgreSQL
- **ORM:** Prisma
- **Authentication:** JWT
- **Frontend:** React Native (Expo)
- **API:** REST (Swagger documented)

1. Architecture Design and System Analysis

This section details the proposed architecture design and system analysis for the Watt Wallet Buddy platform. The aim is to establish a robust, scalable, secure, and maintainable foundation for the fintech-energy mobile application, adhering to best practices in software development and fintech architecture.

1.1. Architectural Principles

To achieve the project's objectives, the following architectural principles will guide the design:

- **Clean Architecture:** Emphasizing separation of concerns, testability, and independence from frameworks, UI, and databases.
- **Scalability:** Designing for horizontal scaling to accommodate a growing user base and transaction volume.

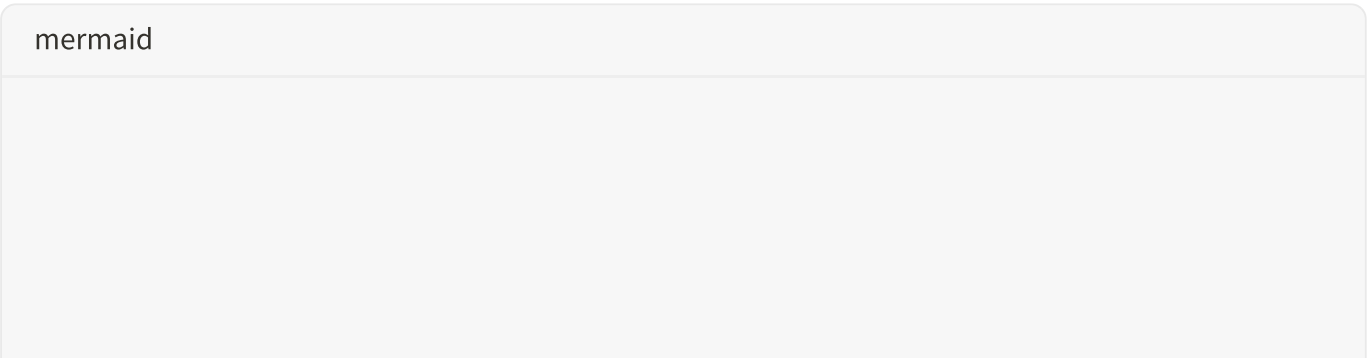
- **Resilience:** Implementing mechanisms to ensure high availability and fault tolerance.
- **Security:** Integrating security measures at every layer, from infrastructure to application code.
- **Maintainability:** Promoting modularity, readability, and ease of modification.
- **Event-Driven Architecture:** Utilizing events for asynchronous communication and decoupling components.

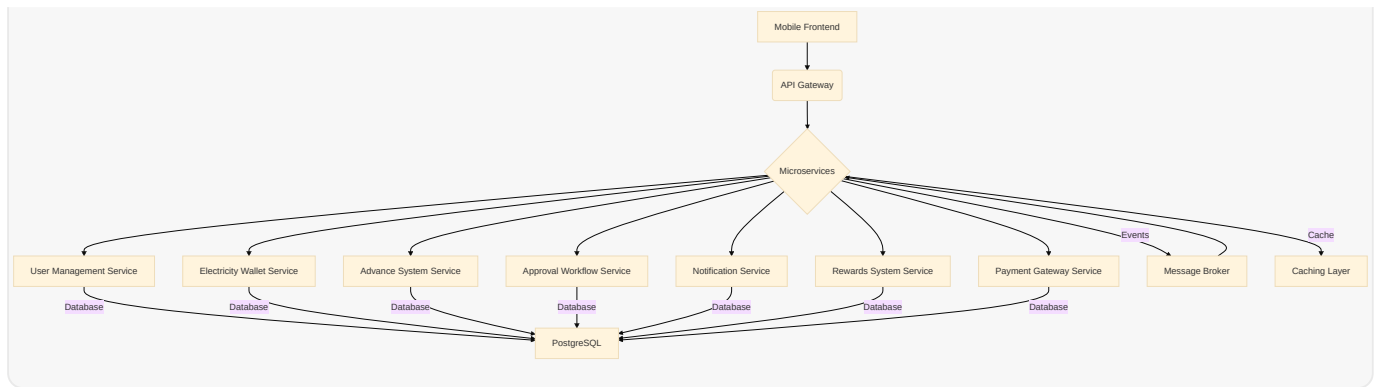
1.2. High-Level System Architecture

The Watt Wallet Buddy platform will adopt a microservices-oriented approach, allowing for independent development, deployment, and scaling of core functionalities. The system will be deployed on a cloud infrastructure, preferably AWS, leveraging managed services for database, messaging, and other components.

1.2.1. Core Components

- **Mobile Frontend (React Native/Expo):** User-facing application for interacting with the platform.
- **API Gateway:** Entry point for all client requests, handling routing, authentication, and rate limiting.
- **Microservices:** Independent services responsible for specific business domains (e.g., User Management, Electricity Wallet, Advance System, Approval Workflow, Notification Service, Rewards System, Payment Gateway).
- **Message Broker (e.g., Kafka, RabbitMQ):** Facilitates asynchronous communication between microservices and enables event-driven patterns.
- **Database (PostgreSQL):** Primary data store for relational data, with potential for specialized databases for specific microservices (e.g., NoSQL for analytics).
- **Caching Layer (e.g., Redis):** Improves performance by storing frequently accessed data.
- **Monitoring & Logging:** Centralized systems for tracking application health, performance, and errors.





1.3. Clean Architecture in Node.js Microservices

Each Node.js microservice will adhere to Clean Architecture principles, organizing code into distinct layers:

- **Entities (Domain Layer):** Contains the core business logic and enterprise-wide business rules. These are plain JavaScript/TypeScript objects representing the application's entities (e.g., User, Meter, Transaction).
- **Use Cases (Application Layer):** Orchestrates the flow of data to and from the entities, implementing application-specific business rules. These use cases interact with the domain layer and orchestrate interactions with external interfaces.
- **Interface Adapters (Infrastructure Layer):** Adapts data from external sources (e.g., web, database, external APIs) to a format suitable for the use cases and entities. This layer includes controllers, presenters, and gateways (repositories).
- **Frameworks & Drivers (External Layer):** Consists of frameworks and tools like Express.js, Prisma, and external services. This layer is the outermost and depends on the inner layers.

1.4. Data Model Review and Enhancements

The existing data models (User, Meter, MeterUser, Transaction, Advance, Approval, Reward, Notification) are a good starting point. They have been reviewed and enhanced to support the proposed architecture and features, ensuring proper relationships, indexing, and data integrity. The detailed Prisma schema is available in the `watt-wallet-buddy-backend/prisma/schema.prisma` file.

2. Backend Implementation: Clean Architecture, RBAC, Approval Workflow, and Payment Abstraction

This section outlines the implementation details for the backend, focusing on a Clean Architecture approach, Role-Based Access Control (RBAC), the Approval Workflow, and a foundation for a Payment Abstraction Layer.

2.1. Project Structure

The backend project (watt-wallet-buddy-backend) is structured as follows to adhere to Clean Architecture principles:

Plain Text

```
watt-wallet-buddy-backend/
├── src/
│   ├── application/                # Application-specific business rules (Use Cases)
│   │   ├── services/
│   │   │   └── AuthService.ts
│   │   └── use-cases/
│   │       ├── LoginUser.ts
│   │       ├── RegisterUser.ts
│   │       ├── RequestAdvance.ts
│   │       └── ReviewAdvance.ts
│   ├── domain/                    # Enterprise-wide business rules (Entities, Repositories)
│   │   ├── entities/
│   │   │   ├── Advance.ts
│   │   │   ├── Approval.ts
│   │   │   ├── Meter.ts
│   │   │   ├── MeterUser.ts
│   │   │   └── User.ts
│   │   └── repositories/
│   │       ├── AdvanceRepository.ts
│   │       ├── ApprovalRepository.ts
│   │       ├── MeterRepository.ts
│   │       ├── MeterUserRepository.ts
│   │       └── UserRepository.ts
│   ├── infrastructure/            # Data access and external services (Prisma Client, External APIs)
│   │   ├── database/
│   │   │   └── prismaClient.ts
│   │   └── repositories/
│   │       ├── PrismaAdvanceRepository.ts
│   │       ├── PrismaApprovalRepository.ts
│   │       ├── PrismaMeterRepository.ts
│   │       ├── PrismaMeterUserRepository.ts
│   │       └── PrismaUserRepository.ts
│   ├── interfaces/                # UI/Web, API, Devices, External Interfaces (Controllers, Middleware)
│   │   ├── controllers/
│   │   │   ├── AdvanceController.ts
│   │   │   └── AuthController.ts
│   │   └── middleware/
│   │       ├── authMiddleware.ts
│   │       └── rbacMiddleware.ts
│   └── index.ts                    # Application entry point
```

```

├── prisma/
│   ├── migrations/           # Prisma migration files
│   └── schema.prisma         # Prisma schema definition
├── .env                       # Environment variables
├── package.json
├── tsconfig.json
└── README.md

```

2.2. Core Feature Implementations

2.2.1. Authentication and Authorization (RBAC)

- **User Entity (`src/domain/entities/User.ts`)**: Defines the `User` interface with properties like `id` , `email` , `password` , `firstName` , `lastName` , and `role` (enum: `PRIMARY` , `SECONDARY` , `ADMIN`).
- **User Repository (`src/domain/repositories/UserRepository.ts`)**: An interface defining methods for user data access (e.g., `findById` , `findByEmail` , `create`).
- **Prisma User Repository (`src/infrastructure/repositories/PrismaUserRepository.ts`)**: Implementation of `UserRepository` using Prisma to interact with the PostgreSQL database.
- **Auth Service (`src/application/services/AuthService.ts`)**: Handles password hashing (using `bcryptjs`) and JWT token generation/verification (using `jsonwebtoken`).
- **Register User Use Case (`src/application/use-cases/RegisterUser.ts`)**: Business logic for user registration, including checking for existing users and hashing passwords.
- **Login User Use Case (`src/application/use-cases/LoginUser.ts`)**: Business logic for user login, comparing passwords and generating JWT tokens.
- **Auth Controller (`src/interfaces/controllers/AuthController.ts`)**: Handles HTTP requests for user registration and login, orchestrating the corresponding use cases.
- **Authentication Middleware (`src/interfaces/middleware/authMiddleware.ts`)**: Verifies JWT tokens from incoming requests and attaches `userId` and `userRole` to the request object.
- **RBAC Middleware (`src/interfaces/middleware/rbacMiddleware.ts`)**: Checks if the authenticated user's role is among the allowed roles for a specific route or action, enforcing Role-Based Access Control.

2.2.2. Approval Workflow (End-to-End)

This critical feature allows secondary users to request electricity advances, which primary users must then approve or decline.

- **Advance Entity** (`src/domain/entities/Advance.ts`): Defines the `Advance` interface, including `userId` , `meterId` , `amount` , `serviceFee` , `vat` , `totalRepay` , `status` (enum: `PENDING` , `APPROVED` , `DECLINED` , `REPAID`), `requestDate` , and `dueDate` .
- **Advance Repository** (`src/domain/repositories/AdvanceRepository.ts`): Interface for advance data access.
- **Prisma Advance Repository** (`src/infrastructure/repositories/PrismaAdvanceRepository.ts`): Prisma implementation for `AdvanceRepository` .
- **Approval Entity** (`src/domain/entities/Approval.ts`): Defines the `Approval` interface, including `advanceId` , `approverId` , `status` (enum: `PENDING` , `APPROVED` , `DECLINED`), `reason` , and `approvedAt` .
- **Approval Repository** (`src/domain/repositories/ApprovalRepository.ts`): Interface for approval data access.
- **Prisma Approval Repository** (`src/infrastructure/repositories/PrismaApprovalRepository.ts`): Prisma implementation for `ApprovalRepository` .
- **Meter Entity** (`src/domain/entities/Meter.ts`): Defines the `Meter` interface, including `id` , `meterNumber` , and `primaryUserId` .
- **Meter Repository** (`src/domain/repositories/MeterRepository.ts`): Interface for meter data access.
- **Prisma Meter Repository** (`src/infrastructure/repositories/PrismaMeterRepository.ts`): Prisma implementation for `MeterRepository` .
- **MeterUser Entity** (`src/domain/entities/MeterUser.ts`): Defines the `MeterUser` interface, linking users to meters and specifying `permissions` .
- **MeterUser Repository** (`src/domain/repositories/MeterUserRepository.ts`): Interface for `MeterUser` data access.
- **Prisma MeterUser Repository** (`src/infrastructure/repositories/PrismaMeterUserRepository.ts`): Prisma implementation for `MeterUser` data access.
- **Request Advance Use Case** (`src/application/use-cases/RequestAdvance.ts`): Contains the business logic for a user to request an advance. It includes checks for user and meter existence, user permissions (primary user or secondary user with `request_advance` permission), and calculates service fees, VAT, and total repayment.
- **Review Advance Use Case** (`src/application/use-cases/ReviewAdvance.ts`): Contains the business logic for a primary user to review an advance request. It verifies the advance status, approver's role (must be `PRIMARY`), and updates the advance status accordingly.

- **Advance Controller** (`src/interfaces/controllers/AdvanceController.ts`): Handles HTTP requests for requesting and reviewing advances.

2.2.3. Payment Abstraction Layer (Foundation)

The payment abstraction layer will be designed to integrate with various payment gateways (e.g., PayFast) without tightly coupling the core application logic to a specific provider. This will involve:

- **Payment Gateway Interface (Domain Layer)**: An interface defining generic payment operations (e.g., `initiatePayment` , `processCallback` , `refund`).
- **Payment Service (Application Layer)**: Orchestrates payment flows, interacting with the `Payment Gateway Interface` .
- **Specific Gateway Implementations (Infrastructure Layer)**: Concrete implementations of the `Payment Gateway Interface` for each payment provider (e.g., `PayFastGateway`).

This modular approach allows for easy switching or adding new payment providers with minimal changes to the core application.

2.2.4. Event-Driven Design (Queues, Messaging)

To support asynchronous operations and decouple services, an event-driven design will be incorporated. This will be particularly useful for notifications, rewards processing, and future integrations.

- **Message Broker Integration**: Utilize a message broker (e.g., RabbitMQ, Kafka, or AWS SQS/SNS) to publish and subscribe to events.
- **Event Publishers**: Services will publish events (e.g., `AdvanceRequested` , `AdvanceApproved` , `PaymentCompleted`) after relevant actions.
- **Event Consumers**: Other services will subscribe to these events to perform their tasks asynchronously (e.g., Notification Service listening for `AdvanceApproved` to send a notification).

3. Frontend Strategy, Security, Scaling, and Future Vision Documentation

This section details the frontend strategy, security and compliance measures, scaling strategy, and the future vision for the Watt Wallet Buddy platform.

3.1. Frontend Strategy (React Native/Expo)

3.1.1. Improved React Native Structure

To enhance maintainability, scalability, and developer experience, the React Native project structure will be refined to follow a modular and feature-based approach. This involves organizing code by feature domains rather than by file type (e.g., all components related to 'Auth' in one folder, all related to 'Meter' in another).

Recommended Structure:

Plain Text

```
src/
├── assets/                # Images, fonts, icons
├── components/            # Reusable UI components (e.g., Button, Card)
├── navigation/           # React Navigation setup (stacks, tabs)
├── screens/              # Top-level screens/pages
│   ├── Auth/             # Authentication-related screens
│   │   ├── LoginScreen.tsx
│   │   └── RegisterScreen.tsx
│   ├── Home/             # Home screen and related components
│   │   └── HomeScreen.tsx
│   └── Meter/            # Meter management screens
│       ├── MeterDetailScreen.tsx
│       └── MeterListScreen.tsx
├── services/             # API calls, external integrations
├── store/                # State management (e.g., Zustand, Redux Toolkit)
├── utils/                # Utility functions, helpers
├── hooks/                # Custom React Hooks
├── themes/               # UI themes, styling configurations
└── App.tsx               # Main application entry point
```

3.1.2. State Management

For effective state management in a complex fintech application, a robust and scalable solution is crucial. While Redux Toolkit is a powerful option, for a project starting with a focus on agility and developer experience, **Zustand** or **React Query** (for server state) combined with React's Context API (for local UI state) can offer a lighter, yet highly capable alternative.

- **Zustand:** A small, fast, and scalable bear-bones state-management solution. It's often preferred for its simplicity and minimal boilerplate compared to Redux, while still offering powerful features like middleware and selectors.
- **React Query:** Excellent for managing server-side data, caching, and synchronization. It significantly reduces the complexity of data fetching, caching, and updating, which is common in data-intensive applications like fintech platforms.

3.1.3. Modern Fintech-Style UI/UX Design

The UI/UX design will prioritize a clean, intuitive, and secure experience, reflecting modern fintech aesthetics. Key design principles include:

- **Simplicity:** Clean layouts, minimal clutter, and clear calls to action.
- **Security Visuals:** Clear indicators for secure transactions, biometric authentication prompts, and privacy controls.
- **Data Visualization:** Intuitive charts and graphs for electricity usage, advance repayment tracking, and rewards.
- **Accessibility:** Ensuring the app is usable by individuals with diverse abilities.
- **Branding:** Consistent use of Watt Wallet Buddy's branding elements (colors, typography, logos).

Key UI/UX Elements:

- **Dashboard:** Overview of electricity balance, recent transactions, advance status, and reward points.
- **Transaction History:** Detailed list of purchases, repayments, and advances with filtering options.
- **Meter Management:** Easy addition, removal, and permission management for shared meters.
- **Approval Workflow:** Clear notification and action flows for primary users to approve/decline advance requests.
- **Gamification:** Visually appealing display of rewards and progress.

3.2. Security & Compliance

Fintech applications demand the highest level of security and compliance. Watt Wallet Buddy will implement a multi-layered security approach.

3.2.1. Authentication & Authorization

- **Multi-Factor Authentication (MFA):** Implement MFA (e.g., SMS OTP, authenticator apps) for critical actions and login.
- **Biometric Authentication:** Support fingerprint and facial recognition for convenient and secure access.
- **JWT (JSON Web Tokens):** Securely transmit user identity and roles between client and server. Tokens will have short lifespans and refresh mechanisms.

- **Role-Based Access Control (RBAC):** Granular control over user permissions based on their assigned roles (PRIMARY, SECONDARY, ADMIN). This has been implemented in the backend architecture.

3.2.2. Data Protection

- **End-to-End Encryption:** Encrypt all data in transit (TLS/SSL) and at rest (database encryption).
- **Data Minimization:** Collect and store only necessary user data.
- **Regular Security Audits:** Conduct periodic penetration testing and vulnerability assessments.
- **Secure Coding Practices:** Adhere to OWASP Top 10 guidelines and secure development lifecycle (SDL).

3.2.3. Fraud Prevention

- **Transaction Monitoring:** Implement real-time transaction monitoring to detect and flag suspicious activities.
- **Rate Limiting:** Prevent brute-force attacks and abuse of API endpoints.
- **Anomaly Detection:** Utilize machine learning to identify unusual usage patterns or advance requests.

3.2.4. Compliance

- **PCI DSS:** If handling card payments directly, ensure compliance with Payment Card Industry Data Security Standard.
- **POPIA (South Africa):** Adhere to the Protection of Personal Information Act for data privacy.
- **Financial Regulations:** Comply with relevant financial regulations in South Africa and any future markets.

3.3. Scaling Strategy

To support thousands of users and future global expansion, Watt Wallet Buddy will adopt a cloud-native, microservices-based scaling strategy.

3.3.1. Microservices Architecture

As outlined in the architecture design, breaking down the monolithic application into independent microservices (e.g., User Service, Meter Service, Advance Service, Payment

Service, Notification Service, Rewards Service) offers several benefits:

- **Independent Deployment:** Services can be deployed and updated independently, reducing downtime and risk.
- **Technology Heterogeneity:** Different services can use different technologies best suited for their specific needs.
- **Improved Fault Isolation:** Failure in one service does not necessarily bring down the entire system.
- **Scalability:** Individual services can be scaled independently based on their load requirements.

3.3.2. Cloud Infrastructure (AWS Recommendation)

Amazon Web Services (AWS) is recommended due to its comprehensive suite of services, scalability, and global reach.

Key AWS Services:

- **Compute:** Amazon EC2 (for container orchestration) or AWS Lambda (for serverless functions for specific tasks).
- **Container Orchestration:** Amazon ECS or Amazon EKS for managing Docker containers of microservices.
- **Database:** Amazon RDS for PostgreSQL for managed relational database services. Amazon DynamoDB for NoSQL use cases (e.g., real-time usage data).
- **Messaging:** Amazon SQS (Simple Queue Service) for message queuing and Amazon SNS (Simple Notification Service) for pub/sub messaging, supporting event-driven architecture.
- **Caching:** Amazon ElastiCache (Redis) for in-memory caching to improve performance.
- **API Gateway:** Amazon API Gateway for managing API traffic, authentication, and rate limiting.
- **Monitoring & Logging:** Amazon CloudWatch for monitoring and logging, AWS X-Ray for distributed tracing.
- **Security:** AWS WAF (Web Application Firewall), AWS Shield, AWS IAM (Identity and Access Management).
- **CI/CD:** AWS CodePipeline, AWS CodeBuild, AWS CodeDeploy for automated deployments.

3.3.3. Database Scaling

- **Read Replicas:** For read-heavy services, use PostgreSQL read replicas to distribute query load.

- **Sharding:** Partitioning data across multiple database instances as the dataset grows.
- **Connection Pooling:** Efficiently manage database connections to reduce overhead.

3.4. Future Vision (Innovation Layer)

The future vision for Watt Wallet Buddy involves integrating advanced technologies to provide a more intelligent and proactive energy management experience.

3.4.1. Smart Meter Integration

- **Auto Electricity Loading:** Directly integrate with smart meters to automatically load purchased electricity tokens, eliminating manual input.
- **Real-time Usage Monitoring:** Provide users with real-time data on their electricity consumption, enabling better budget management.
- **Socket-Level Electricity Tracking:** (Advanced) Integrate with smart plugs or IoT devices to monitor electricity usage at the appliance level, offering granular insights.

3.4.2. AI-Based Electricity Usage Predictions

- **Predictive Analytics:** Utilize machine learning models to analyze historical usage data, weather patterns, and user behavior to predict future electricity consumption.
- **Personalized Recommendations:** Offer tailored advice on energy saving, optimal purchase times, and advance amounts based on predicted usage.
- **Proactive Alerts:** Send smart alerts for potential low electricity based on predicted consumption, rather than just current balance.

3.4.3. Event-Driven Architecture for Innovation

The event-driven architecture, facilitated by a message broker, will be crucial for integrating these future innovations. New services (e.g., Smart Meter Service, AI Prediction Service) can publish and subscribe to events without tightly coupling with existing services, allowing for rapid iteration and expansion of features.

This comprehensive approach to frontend strategy, security, scaling, and future vision will position Watt Wallet Buddy as a leading, innovative, and investor-ready platform in the fintech-energy sector.

4. References

[1] Redpanda. (n.d.). 5 best practices for building scalable FinTech systems. Retrieved from

- [2] SDK.finance. (2025, April 10). Fundamentals of FinTech Architecture: Challenges, and Solutions. Retrieved from
- [3] Medium. (n.d.). A definitive guide to building a NodeJS app, using Clean Architecture and TypeScript. Retrieved from
- [4] Dev.to. (n.d.). Clean Architecture in Node.js APIs: Mastering Separation of Concerns. Retrieved from
- [5] Medium. (n.d.). Clean Architecture in Node.js. Introduction. Retrieved from
- [6] Mobidev. (n.d.). How to Build a Fintech App: Approach, Architecture, and Scalability. Retrieved from
- [7] Trio.dev. (n.d.). Microservices Architecture for Fintech in 2026. Retrieved from
- [8] DevOps.com. (n.d.). The Future of Scalable Digital Architecture in Fintech. Retrieved from
- [9] Academia.edu. (2021). Cloud Native Architecture for Scalable Fintech Applications with Real Time Payments. Retrieved from
- [10] ResearchGate. (2024). API integration in FinTech: Challenges and best practices. Retrieved from
- [11] ResearchGate. (2025). Architecting Scalable and Resilient Fintech Platforms with AI/ML Integration. Retrieved from
- [12] OAPUB. (2025). ARCHITECTURAL AUDIT OF HIGH-LOAD FINTECH SYSTEMS: ASSESSMENT METHODOLOGIES, COMMON VULNERABILITIES, AND REFACTORING. Retrieved from
- [13] authors.go2articles.com. (2025). The role of serverless architectures in revolutionizing FinTech solutions. Retrieved from
- [14] Gapsystudio. (2026, January 28). How to Architect a Scalable Fintech Design System. Retrieved from
- [15] Medium. (2025, May 24). Best Practices for Architectural Decision-Making in Fintech. Retrieved from
- [16] Fintechweekly. (2026, February 19). How to Implement Zero-Trust Architecture in Fintech Applications. Retrieved from
- [17] LinkedIn. (n.d.). Building CLEAN Architecture in Node.js: Part 1. Retrieved from
- [18] GitHub. (n.d.). javierteloz/clean-architecture-nodejs. Retrieved from
- [19] Google Books. (2016). Node.js Design Patterns. Retrieved from
- [20] Google Books. (2025). Node.js: the comprehensive guide. Retrieved from
- [21] ResearchGate. (n.d.). Propuesta de implementación de arquitectura Clean para proyectos Node.js. Retrieved from